UniEVANGÉLICA

Programação Orientada a Objetos

Documentação técnica do projeto

Carteira Fintech

Spring Boot · PostgreSQL · Thymeleaf

Interfaces, Design Patterns, SOLID, Clean Code e Segurança

Material de apoio · acompanha o repositório do projeto no GitHub

VISÃO GERAL

O que é este projeto

A **Carteira Fintech** e uma aplicacao web simples que processa pagamentos por **Pix, cartao e boleto**. Ela foi construida para ensinar, na pratica, o conteudo da Semana 11 (interfaces em Java), aplicando as boas praticas que o mercado espera de um sistema real: separacao em camadas, padroes de projeto, principios SOLID, Clean Code e seguranca da informacao. O codigo usa recursos simples da linguagem, de proposito, para acompanhar o nivel da turma.

ARQUITETURA

Organização em camadas

Cada camada tem uma unica responsabilidade e conhece apenas a camada vizinha. Isso mantem o sistema organizado e facil de evoluir — e e a base do principio da Responsabilidade Unica (SRP).

web/ — apresentação

Controllers (Spring MVC) e o DTO do formulario. Recebem a requisicao HTTP, validam a entrada e chamam o servico. Nao contem regra de negocio.

service/ — negócio

Regras da aplicacao. O `TransacaoService` orquestra a fabrica e o repositorio. Depende de **interfaces**, nunca de classes concretas.

pagamento/ — domínio

O coracao da aula: a interface `MeioDePagamento`, suas implementacoes (Pix, Cartao, Boleto) e a fabrica. E aqui que moram os conceitos da Semana 11.

model/ + repository/ — dados

Entidades JPA (`Usuario`, `Transacao`) e os repositorios Spring Data que leem e gravam no PostgreSQL.

Fluxo de um pagamento: o navegador envia o formulario → **PagamentoController** valida → **TransacaoService** pede a estrategia certa à **PagamentoFactory** → chama `pagar()` pelo contrato → grava a **Transacao** pelo **Repository** → a pagina mostra o comprovante. Nenhuma camada precisa saber qual meio de pagamento foi usado.

CONTEÚDO DA AULA

Onde a Semana 11 aparece no código

Conceito	Arquivo	Como aparece
Interface / contrato	MeioDePagamento.java	Lista pagar(), getTipo() e um metodo default rotulo().
implements / polimorfismo	Pix, Cartao, Boleto	Tres classes cumprem o mesmo contrato de formas diferentes.
Implementação múltipla	Pix/Cartao/Boleto + Auditavel	Cada classe implementa MeioDePagamento E Auditavel.
Método default (Java 8)	MeioDePagamento.rotulo()	Metodo com corpo na interface, herdado por todas.
Programar para a interface	TransacaoServiceImpl	Depende de PagamentoFactory e do repositorio (abstracoes).
Interface do framework	UsuarioDetailsService	Implementa UserDetailsService para o login do Spring.

O contrato e uma de suas implementacoes:

implementação múltipla

```
public interface MeioDePagamento {
ResultadoPagamento pagar(BigDecimal valor);
TipoPagamento getTipo();
default String rotulo() { return "Pagamento via " + getTipo(); }
}
public class Pix implements MeioDePagamento, Auditavel {
public ResultadoPagamento pagar(BigDecimal valor) { /* ... */ }
public String descricaoAuditoria() { return "Pagamento via Pix"; }
}
```

PADRÕES DE PROJETO

Design Patterns usados

Padroes de projeto sao solucoes testadas para problemas que se repetem. Este projeto usa tres, todos apoiados em interfaces:

1. Strategy (Estratégia)

Cada meio de pagamento e uma "estrategia" intercambiavel por tras da interface `MeioDePagamento`. Trocar a forma de pagar e trocar a estrategia, sem mexer em quem a usa. **Problema que resolve:** escolher um entre varios comportamentos em tempo de execucao.

2. Factory Method (Fábrica)

A `PagamentoFactory` decide qual estrategia criar a partir do `TipoPagamento`. Concentra o `new` em um unico lugar; o resto do sistema recebe sempre o tipo da interface. **Problema que resolve:** criar objetos sem espalhar `new Pix()` pelo codigo.

3. Repository (Repositório)

`UsuarioRepository` e `TransacaoRepository` sao interfaces que o Spring Data implementa sozinho. Escondem o acesso ao banco atras de metodos simples. **Problema que resolve:** separar a regra de negocio dos detalhes de persistencia.

PagamentoFactory.java

```
// Factory: cria a estrategia certa, devolvida pelo tipo do contrato
public MeioDePagamento criar(TipoPagamento tipo) {
switch (tipo) {
case PIX: return new Pix();
case CARTAO: return new Cartao();
case BOLETO: return new Boleto();
default: throw new IllegalArgumentException(...);
}
}
```

PRINCÍPIOS

SOLID no projeto

SOLID sao cinco principios que deixam o codigo flexivel e facil de manter. Veja onde cada um aparece:

Princípio	Significado	No projeto
S — Responsabilidade Única	Cada classe tem um motivo para mudar.	Controller valida, Service decide, Repository persiste.
O — Aberto/Fechado	Aberto a extensão, fechado a modificação.	Novo meio de pagamento = nova classe; Service não muda.
L — Substituição de Liskov	Subtipos substituem o tipo base sem quebrar.	Qualquer MeioDePagamento funciona onde o contrato é esperado.
I — Segregação de Interfaces	Contratos pequenos e coesos.	MeioDePagamento e Auditavel são separados, não uma só interface.
D — Inversão de Dependência	Dependa de abstrações, não de implementações.	TransacaoServiceImpl recebe abstrações pelo construtor.

O princípio Aberto/Fechado na prática: para a carteira passar a aceitar PayPal, basta criar `class PayPal implements MeioDePagamento, Auditavel` e adicionar um `case` na fábrica. O controller, o service e as páginas **não mudam uma linha** — é o retorno de ter programado para interfaces.

Observe que SOLID não é teoria solta: cada princípio se apoia no mesmo recurso que a aula apresentou — a interface. Por isso interfaces são consideradas a porta de entrada para projetar bom software, e não apenas escrever código que funciona.

CLEAN CODE

Código limpo aplicado

- **Nomes que revelam a intenção** — `processarPagamento`, `MeioDePagamento`, `descricaoAuditoria`.
- **Métodos curtos**, cada um fazendo uma coisa só.
- **Sem números mágicos** — o teto de R$ 50.000 fica declarado na validação, com mensagem clara.
- **Objetos imutáveis** quando possível (`ResultadoPagamento`).
- **Comentários explicam o "porquê"**, não o óbvio; o código conta o "o quê".

SEGURANÇA DA INFORMAÇÃO

Boas práticas e o porquê de cada uma

Prática	Por quê
Senhas com hash BCrypt	Se o banco vazar, as senhas não são reveladas; BCrypt é lento e usa sal aleatório.
Autenticação + autorização	Só usuários logados acessam o sistema (Spring Security).
Proteção CSRF ativada	Impede que outro site force uma ação em nome do usuário logado.
Validação de entrada	Bean Validation barra dados inválidos antes da regra de negócio.
Consultas parametrizadas	Spring Data/JPA previne injeção de SQL automaticamente.
Segredos em variáveis de ambiente	Senha do banco fora do código; .env no .gitignore.
Mascaramento de dados	O cartão exibe só os 4 últimos dígitos; nada sensível em log.
BigDecimal para dinheiro	Evita erros de arredondamento do double em valores financeiros.

EXECUÇÃO

Como rodar o projeto

O projeto acompanha o Maven Wrapper (`mvnw`), que baixa o Maven automaticamente — basta ter o Java 17+ instalado.

Opção A — sem instalar nada (H2)

```
./mvnw spring-boot:run
-Dspring-boot.run.profiles=h2
```

Banco em memória, zero configuração. Abra `localhost:8080` e entre com **aluno / senha123**.

Opção B — com PostgreSQL

```
docker compose up -d
cp .env.example .env
./mvnw spring-boot:run
```

Mais próximo de produção; o banco roda em container.

Compartilhar no GitHub: o repositório já traz `.gitignore` (ignora `target/` e `.env`), `README.md`, licença MIT e o Maven Wrapper. Basta `git init`, `git add .`, `git commit` e `git push` para o seu repositório.

Referências

DEITEL, P.; DEITEL, H. *Java: como programar*. 10. ed. Pearson, 2017 — cap. 10 (interfaces).

BLOCH, J. *Effective Java*. 3. ed. Addison-Wesley, 2018.

MARTIN, R. C. *Código Limpo (Clean Code)*. Alta Books, 2009; e os princípios SOLID.

GAMMA, E. et al. *Padrões de Projeto (Design Patterns)*. Bookman, 2000 — Strategy, Factory.

VMware. *Spring Boot Reference Documentation*. Disponível em: spring.io.

OWASP. *OWASP Top Ten* — principais riscos de segurança em aplicações web. Disponível em: owasp.org.